

Recursividade

A recursividade é um conceito fundamental em matemática e ciência da computação que envolve a definição de uma função ou algoritmo em termos de si mesmo. Em termos de estruturas de dados, a recursividade pode ser usada para percorrer, manipular ou analisar essas estruturas de uma maneira elegante e eficiente. Através do uso da recursividade, é possível simplificar a lógica e a implementação de problemas complexos, tornando-os mais acessíveis e compreensíveis.

Uma solução recursiva geralmente consiste em duas partes básicas:

Caso(s) base: O(s) caso(s) base é(são) a(s) condição(ões) mais simples do problema que pode(m) ser resolvida(s) diretamente, sem a necessidade de chamadas recursivas adicionais. O caso base serve como ponto de partida e de término para a recursão, evitando que a função entre em um loop infinito de chamadas recursivas. Em geral, o caso base é definido para os valores mais simples do argumento da função.

Passo(s) recursivo(s): O(s) passo(s) recursivo(s) é(são) a(s) parte(s) do algoritmo onde a função chama a si mesma com argumentos menores, simplificados ou modificados, com o objetivo de resolver o problema maior em termos de problemas menores e mais próximos do caso base. A recursão permite dividir o problema em subproblemas menores, e a solução para o problema original é construída a partir das soluções dos subproblemas.

Inicialmente, exploraremos a recursividade a partir de exemplos matemáticos, começando pelo cálculo do fatorial.

Fatorial

O fatorial de um número inteiro não negativo n (denotado como $n!$) é o produto de todos os números inteiros positivos menores ou iguais a n . Ou seja, $n! = n \times (n-1) \times (n-2) \times \dots \times 1$. Por definição, o fatorial de 0 é 1 ($0! = 1$).

A recursividade pode ser aplicada no cálculo do fatorial, como segue:

Caso base: Se $n = 0$, então $n! = 1$. O fatorial de 0 é definido como 1, e não é necessário realizar nenhuma chamada recursiva adicional.

Passo recursivo: Se $n > 0$, então $n! = n \times (n-1)!$. Neste passo, a função fatorial chama a si mesma com o argumento $(n-1)$, que é um valor menor do que n . A solução do fatorial de n é construída a partir da solução do fatorial de $(n-1)$.

Observe que no passo recursivo, a função fatorial é definida em termos de si mesma, com o argumento $(n-1)$. Essa definição recursiva permite simplificar o cálculo do fatorial e proporciona uma base sólida para a compreensão de outros exemplos matemáticos que envolvem recursividade.

Mas vamos ver como chegamos a estas duas partes básicas no caso do cálculo do fatorial, exemplificando com o cálculo do fatorial de 5.

Sabemos então que:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1 \text{ (por definição)}$$

Vamos ver como se dá esta relação de um fatorial a outro:

$$5! = 5 \times \boxed{4 \times 3 \times 2 \times 1}$$

$$4! = \boxed{4 \times 3 \times 2 \times 1}$$

Como pode-se ver, a sequência $4 \times 3 \times 2 \times 1$ que multiplica por 5, da primeira linha, equivale ao $4!$, conforme a segunda linha.

Logo:

$$\mathbf{5! = 5 \times 4!}$$

Prosseguindo:

$$4! = 4 \times \boxed{3 \times 2 \times 1}$$

$$3! = \boxed{3 \times 2 \times 1}$$

De igual forma, tem-se que a sequência $3 \times 2 \times 1$ que multiplica por 4, na primeira linha, equivale ao $3!$, da segunda linha.

Então, vê-se que:

$$\mathbf{4! = 4 \times 3!}$$

Mais uma vez:

$$3! = 3 \times \boxed{2 \times 1}$$

$$2! = \boxed{2 \times 1}$$

Assim, tem-se que a sequência 2×1 que multiplica por 3, na primeira linha, equivale ao $2!$, da segunda linha.

Novamente, vê-se que:

$$3! = 3 \times 2!$$

Uma vez mais:

$$2! = 2 \times 1$$

$$1! = 1$$

Fechando, então, temos:

$$2! = 2 \times 1!$$

E, por fim:

$$1! = 1$$

$$0! = 1 \text{ (por definição)}$$

Juntando tudo, podemos mostrar que:

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1!$$

$$1! = 1$$

$$0! = 1$$

Vamos agora substituir o símbolo matemático do fatorial (!) por uma chamada de função. O mesmo conjunto de cálculos ficaria assim:

$$\text{fat}(5) = 5 \times \text{fat}(4)$$

$$\text{fat}(4) = 4 \times \text{fat}(3)$$

$$\text{fat}(3) = 3 \times \text{fat}(2)$$

$$\text{fat}(2) = 2 \times \text{fat}(1)$$

$$\text{fat}(1) = 1$$

$$\text{fat}(0) = 1$$

Se pegarmos a primeira linha, para exemplificar, e substituirmos o valor do argumento por n , temos a seguinte generalização:

$$\text{fat}(5) = 5 \times \text{fat}(4)$$

$$\text{fat}(n) = n \times \text{fat}(n-1)$$

Isso só não acontece quando n é igual a 1 ou a 0, quando o fatorial será igual a 1:

$$\text{fat}(1) = 1$$

$$\text{fat}(0) = 1$$

Assim, conseguimos visualizar o **passo recursivo** (quando a função chama a si mesmo): $\text{fat}(n)$

$= n \times \text{fat}(n-1)$, e **os casos base** (os casos em que há a parada de chamada), que são quando n é igual a 1 ou a 0.

Se pensarmos em termos de condicionais, temos que se n é igual a 1 ou a 0 o valor do fatorial será igual a 1 e, caso seja maior do que 1, será igual a $n \times \text{fat}(n-1)$. Em código Python isso tudo virará:

```
def fat(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * fat(n - 1)
```

Neste código, a função `fat(n)` verifica se n é igual a 0 ou 1 (caso base) e retorna 1 nesse caso. Se n for maior do que 1, a função chama a si mesma com o argumento $n - 1$ (passo recursivo) e multiplica o resultado pelo valor de n .

Aproveitaremos a função `fat` para mostrar como isso acontece, efetivamente, na execução do código. Seu funcionamento está relacionado a uma estrutura interna das linguagens de programação que mantém o controle das chamadas de função e seus respectivos contextos. Quando uma função chama a si mesma recursivamente, cada chamada cria um novo quadro de ativação na pilha, que contém informações sobre a chamada atual da função, como valores dos argumentos, variáveis locais e o endereço de retorno. Isso permite que cada chamada recursiva tenha seu próprio contexto e possa ser resolvida corretamente.

Vamos ver como isso funciona para a chamada da função `fat(3)`.

No programa principal, temos a chamada `print(fat(3))`. Para que a função `print` possa imprimir o resultado, a função `fat` deve ser executada e finalizada primeiro.

Podemos visualizar a execução do código do programa principal em uma área de memória específica dela:

```
print(fat(3))
```

Quando a função `fat` é chamada com o argumento 3, um quadro é criado na pilha de chamadas, armazenando informações sobre essa chamada específica, como o valor de `n` (neste caso, 3). O programa principal espera a execução da função `fat` ser concluída.

<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 3
<pre>print(fat(3))</pre>	

A função `fat` verifica se `n` é igual a 0 ou 1, e como `n` é igual a 3, ela executa a linha `return n * fat(n - 1)`, chamando a função `fat` novamente com o argumento `n - 1` (ou seja, 2). A chamada original da função `fat` (com `n` igual a 3) aguarda a conclusão desta nova chamada.

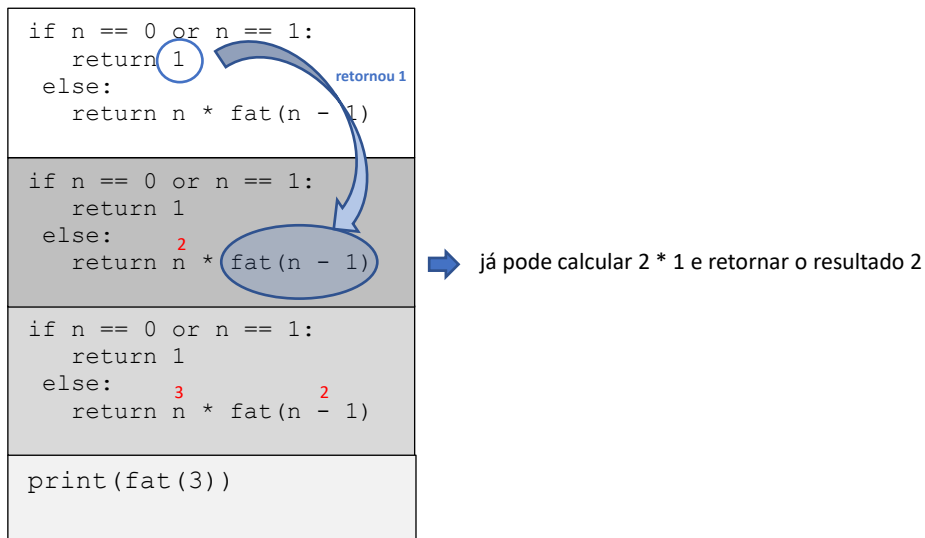
<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 2
<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 3
<pre>print(fat(3))</pre>	

A função `fat` é chamada novamente com o argumento 2. Um novo quadro é criado na pilha de chamadas, armazenando o valor atual de `n` (neste caso, 2). A chamada anterior da função `fat` (com `n` igual a 3) ainda está na pilha, aguardando a conclusão desta chamada.

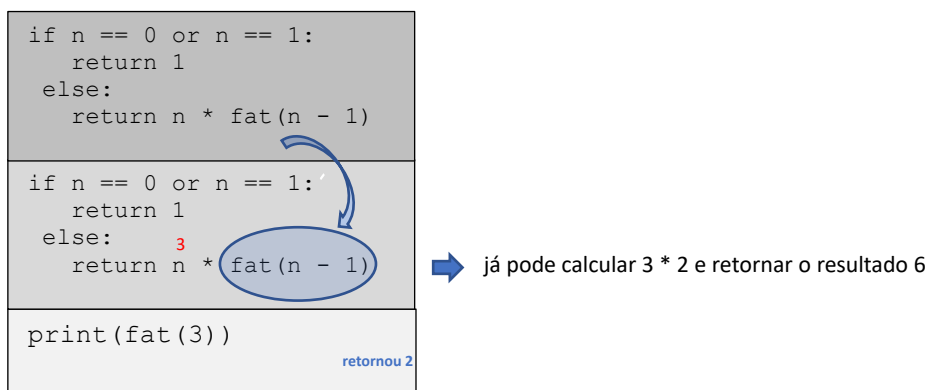
Como `n` não é igual a 0 ou 1 novamente, a função `fat` executa a linha `return n * fat(n - 1)` com `n` valendo 2. A função `fat` é chamada mais uma vez, agora com o argumento `n - 1` (que é igual a 1). A chamada anterior da função `fat` (com `n` igual a 2) aguarda a conclusão desta nova chamada.

<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 1
<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 2
<pre>if n == 0 or n == 1: return 1 else: return n * fat(n - 1)</pre>	n = 3
<pre>print(fat(3))</pre>	

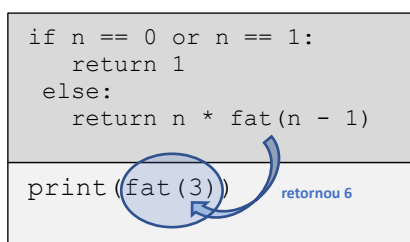
A função `fat` é chamada pela terceira vez com o argumento 1. Como `n` é igual a 1, a função entra no caso base e retorna o valor 1. Este valor retornado será utilizado pela chamada anterior da função `fat` (com `n` igual a 2) para continuar o cálculo.



A chamada da função `fat` que retornou 1 é, então, encerrada e removida da pilha de recursividade.



A chamada da função `fat` com `n` igual a 2 agora pode continuar o cálculo, multiplicando 2 pelo valor retornado (1) e retornando o resultado (2). Essa chamada também é encerrada e removida da pilha de recursividade.



Por último, a chamada da função `fat` com `n` igual a 3 pode continuar o cálculo, multiplicando 3 pelo valor retornado (2) e retornando o resultado (6). A chamada da função `fat` com `n` igual a 3 é encerrada e removida da pilha de recursividade, e o valor 6 é retornado para o programa principal.

```
print(fat(3))
```

A função `print` agora pode imprimir o resultado do fatorial de 3, que é 6.

Vamos ver agora outros exemplos para entender um pouco mais de recursividade.

Multiplicação de dois números inteiros

A multiplicação de um número inteiro 'a' por outro número inteiro 'b' (denotado como `mult(a, b)`) pode ser representada como a soma de 'a' parcelas iguais a 'b'. Por exemplo, `mult(5, 3) = 3 + 3 + 3 + 3 + 3` e `mult(4, 3) = 3 + 3 + 3 + 3`. A recursividade pode ser aplicada no cálculo da multiplicação, como segue:

Casos base:

Se `a = 0`, então `mult(a, b) = 0`, pois qualquer número multiplicado por 0 é 0.

Se `a = 1`, então `mult(a, b) = b`, pois qualquer número multiplicado por 1 é igual a ele mesmo.

Passo recursivo: Se `a > 1`, então `mult(a, b) = b + mult(a-1, b)`. Neste passo, a função de multiplicação chama a si mesma com o argumento `(a-1)`, que é um valor menor do que 'a'. A solução da multiplicação de 'a' por 'b' é construída a partir da solução da multiplicação de `(a-1)` por 'b'.

Observe que, no passo recursivo, a função de multiplicação é definida em termos de si mesma, com o argumento `(a-1)`. Essa definição recursiva está longe de simplificar o cálculo da multiplicação, mas proporciona uma base sólida para a compreensão de outros exemplos matemáticos que envolvem recursividade.

Vamos ver como chegamos a essas duas partes básicas no caso do cálculo da multiplicação, exemplificando com o cálculo de `5x3`:

Sabemos então que:

$$5 \times 3 = 3 + 3 + 3 + 3 + 3$$

$$4 \times 3 = 3 + 3 + 3 + 3$$

$$3 \times 3 = 3 + 3 + 3$$

$$2 \times 3 = 3 + 3$$

$$1 \times 3 = 3 \text{ (caso base)}$$

$$0 \times 3 = 0 \text{ (caso base)}$$

Vamos ver como se dá a relação entre a multiplicação de um número por outro e as chamadas recursivas:

$$5 \times 3 = 3 + \boxed{3 + 3 + 3 + 3}$$

$$4 \times 3 = \boxed{3 + 3 + 3 + 3}$$

Como pode-se ver, a soma de 3 quatro vezes, da segunda linha, quando somada mais uma vez com 3, resulta no que está apresentado na primeira linha.

Logo:

$$\mathbf{5 \times 3 = 3 + 4 \times 3}$$

Prosseguindo:

$$4 \times 3 = 3 + \boxed{3 + 3 + 3}$$

$$3 \times 3 = \boxed{3 + 3 + 3}$$

De igual forma, tem-se que a soma de 3 três vezes, na segunda linha, quando somada mais uma vez com 3, resulta na primeira linha.

Então, vê-se que:

$$\mathbf{4 \times 3 = 3 + 3 \times 3}$$

Mais uma vez:

$$3 \times 3 = 3 + \boxed{3 + 3}$$

$$2 \times 3 = \boxed{3 + 3}$$

Assim, tem-se que a soma de 3 duas vezes, na segunda linha, quando somada mais uma vez com 3, resulta na primeira linha.

Novamente, vê-se que:

$$3 \times 3 = 3 + 2 \times 3$$

Uma vez mais:

$$2 \times 3 = 3 + 3$$

$$1 \times 3 = 3$$

Fechando, então, temos:

$$2 \times 3 = 3 + 1 \times 3$$

E, por fim, temos os casos base:

$$1 \times 3 = 3$$

$$0 \times 3 = 0$$

Juntando tudo, podemos mostrar que:

$$5 \times 3 = 3 + 4 \times 3$$

$$4 \times 3 = 3 + 3 \times 3$$

$$3 \times 3 = 3 + 2 \times 3$$

$$2 \times 3 = 3 + 1 \times 3$$

$$1 \times 3 = 3$$

$$0 \times 3 = 0$$

Digamos que não temos a operação multiplicação implementada, que aquele “x” ali é só uma expressão matemática que não existe no Python. Então poderíamos substituir sua apresentação por uma função correspondente, que chamaremos de mult.

Desta forma, trocando o símbolo “x” pela função mult, teremos:

$$\text{mult}(5, 3) = 3 + \text{mult}(4, 3)$$

$$\text{mult}(4, 3) = 3 + \text{mult}(3, 3)$$

$$\text{mult}(3, 3) = 3 + \text{mult}(2, 3)$$

$$\text{mult}(2, 3) = 3 + \text{mult}(1, 3)$$

$$\text{mult}(1, 3) = 3$$

mult(0, 3) = 0

Se pegarmos a primeira linha, para exemplificar, e substituirmos o valor dos argumentos por a e b, temos a seguinte generalização:

mult(5, 3) = 3 + mult(4, 3)
↓ ↓ ↓ ↓ ↓
mult(a, b) = b + mult(a-1, b)

Isso só não acontece quando a é igual a 1 ou a 0, quando a multiplicação será igual a b e 0, respectivamente:

mult(1, b) = b

mult(0, b) = 0

Assim, conseguimos visualizar o passo recursivo (quando a função chama a si mesma): $\text{mult}(a, b) = b + \text{mult}(a-1, b)$, e os casos base (os casos em que há a parada de chamada), que são quando a é igual a 1 ou a 0.

Se pensarmos em termos de condicionais, temos que se a é igual a 1, o valor da multiplicação será igual a b; se a é igual a 0, o valor da multiplicação será igual a 0; e, caso 'a' seja maior do que 1, o valor da multiplicação será igual a $b + \text{mult}(a-1, b)$. Em código Python, isso se traduziria em:

```
def mult(a, b):  
    if a == 0:  
        return 0  
    elif a == 1:  
        return b  
    else:  
        return b + mult(a - 1, b)
```

Neste código, a função `mult(a, b)` verifica se a é igual a 0 (caso base) e retorna 0 nesse caso. Se a for igual a 1 (outro caso base), a função retorna b. Se a for maior do que 1, a função chama a si mesma com o argumento a - 1 (passo recursivo) e soma o resultado ao valor de b.

Sequência de Fibonacci

A sequência de Fibonacci é uma série de números na qual cada número é a soma dos dois anteriores. Os primeiros números da sequência são: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... e assim por diante. A recursividade pode ser aplicada no cálculo da sequência de Fibonacci, como segue:

Caso base: Se $n = 0$, então $\text{fib}(n) = 0$. Se $n = 1$, então $\text{fib}(n) = 1$. Nesses casos, não é necessário realizar nenhuma chamada recursiva adicional.

Passo recursivo: Se $n > 1$, então $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$. Neste passo, a função Fibonacci chama a si mesma com os argumentos $(n-1)$ e $(n-2)$, que são valores menores do que n . A solução do n -ésimo termo da sequência de Fibonacci é construída a partir da solução dos termos $(n-1)$ e $(n-2)$.

Observe que no passo recursivo, a função Fibonacci é definida em termos de si mesma, com os argumentos passados como $(n-1)$ e $(n-2)$. Essa definição recursiva permite simplificar o cálculo da sequência de Fibonacci e proporciona uma base sólida para a compreensão de outros exemplos matemáticos que envolvem recursividade.

Mas vamos ver como chegamos a estas duas partes básicas no caso do cálculo da sequência de Fibonacci, exemplificando com o cálculo do quinto termo da sequência. Sabemos então que:

$$\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$$

$$\text{fib}(4) = \text{fib}(3) + \text{fib}(2)$$

$$\text{fib}(3) = \text{fib}(2) + \text{fib}(1)$$

$$\text{fib}(2) = \text{fib}(1) + \text{fib}(0)$$

$$\text{fib}(1) = 1$$

$$\text{fib}(0) = 0$$

Se pegarmos a primeira linha, para exemplificar, e substituirmos o valor do argumento por n , temos a seguinte generalização:

$$\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$$


$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Isso só não acontece quando n é igual a 0 ou a 1, quando a sequência de Fibonacci será igual a n :

$$\text{fib}(1) = 1$$

$$\text{fib}(0) = 0$$

Assim, conseguimos visualizar o passo recursivo (quando a função chama a si mesma): $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, e os casos base (os casos em que há a parada de chamada), que são quando n é igual a 0 ou a 1.

Se pensarmos em termos de condicionais, temos que se n é igual a 0 ou a 1, o valor da sequência de Fibonacci será igual a n e, caso seja maior do que 1, será igual a $\text{fib}(n-1) + \text{fib}(n-2)$. Em código Python isso tudo virará:

```
def fib(n):  
    if n == 0 or n == 1:  
        return n
```

```
else:  
    return fib(n - 1) + fib(n - 2)
```

Neste código, a função `fib(n)` verifica se `n` é igual a 0 ou 1 (caso base) e retorna `n` nesse caso. Se `n` for maior do que 1, a função chama a si mesma com os argumentos `n - 1` e `n - 2` (passo recursivo) e soma os resultados.

Torre de Hanoi

A lenda da Torre de Hanoi é uma história fictícia que envolve um antigo templo, monges e um quebra-cabeça matemático. A história foi criada pelo matemático francês Edouard Lucas em 1883, quando ele inventou o quebra-cabeça da Torre de Hanoi. A lenda serve como uma interessante ilustração do problema e de seu contexto histórico e cultural.

Segundo a lenda, em um antigo templo na cidade de Benares, na Índia, existia uma sala com três hastes de diamante fixas no chão. No início dos tempos, os monges do templo receberam 64 discos de ouro, todos de diferentes tamanhos, e foram instruídos a colocá-los em uma das hastes. Os discos foram inicialmente organizados em ordem decrescente de tamanho, com o maior disco na base e o menor disco no topo.

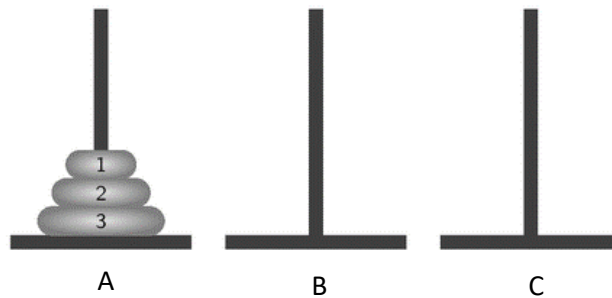
Os monges foram então incumbidos de uma tarefa divina: mover todos os 64 discos de ouro para outra haste, seguindo as regras que agora conhecemos como as regras da Torre de Hanoi:

- Apenas um disco pode ser movido de cada vez.
- Cada movimento consiste em pegar o disco superior de uma das hastes e colocá-lo na parte superior de outra haste.
- Um disco não pode ser colocado em cima de outro disco menor.

A lenda diz que, uma vez que os monges completassem a tarefa, o templo se transformaria em pó e o mundo chegaria ao fim. Essa história ressalta a dificuldade e a complexidade do problema da Torre de Hanoi, mesmo para um número relativamente pequeno de discos.

Na realidade, a quantidade de movimentos necessários para resolver a Torre de Hanoi com 64 discos é $2^{64} - 1$, o que equivale a 18.446.744.073.709.551.615 movimentos. Mesmo se os monges conseguissem fazer um movimento por segundo, levaria mais de 584 bilhões de anos para completar a tarefa - um período de tempo muito maior do que a história registrada da humanidade.

Vamos considerar uma Torre de Hanoi com 3 discos, para manter a explicação simples e ilustrar a recursividade envolvida na solução do problema. Temos três hastes: A (origem), B (auxiliar) e C (destino). Inicialmente, todos os discos estão na haste A, em ordem decrescente de tamanho, com o maior na base e o menor no topo.



O objetivo é mover todos os discos da haste A para a haste C, seguindo as regras da Torre de Hanoi mencionadas anteriormente. Aqui está a solução passo a passo e como a recursividade está envolvida:

1. Mova o menor disco (disco 1) de A para C. Agora temos discos 2 e 3 na haste A e disco 1 na haste C.
2. Mova o disco do meio (disco 2) de A para B. Agora temos disco 3 na haste A, disco 2 na haste B e disco 1 na haste C.
3. Mova o menor disco (disco 1) de C para B. Agora temos o disco 3 na haste A e discos 1 e 2 na haste B, em ordem correta.

Agora, nosso objetivo é mover os discos 1 e 2 da haste B para a haste C, usando a haste A como auxiliar. Essencialmente, estamos resolvendo um subproblema menor da Torre de Hanoi, com 2 discos em vez de 3. Observe que isso é uma instância recursiva do problema original.

4. Mova o maior disco (disco 3) de A para C. Agora temos discos 1 e 2 na haste B e disco 3 na haste C.
5. Mova o menor disco (disco 1) de B para A. Agora temos disco 1 na haste A, disco 2 na haste B e disco 3 na haste C.
6. Mova o disco do meio (disco 2) de B para C. Agora temos disco 1 na haste A e discos 2 e 3 na haste C, com o disco 2 no topo do disco 3.
7. Por fim, mova o menor disco (disco 1) de A para C. Todos os três discos estão agora na haste C, em ordem decrescente de tamanho, e o problema foi resolvido.

A recursividade aqui é evidente no fato de que resolvemos um problema menor da Torre de Hanoi (com 2 discos) como parte da solução do problema original (com 3 discos).

Vamos ver então como chegamos às duas partes básicas no caso do cálculo da Torre de Hanoi, exemplificando com 3 discos:

Caso base: Se $n = 1$ (apenas um disco), mova o disco diretamente para a haste de destino.

Passo recursivo: Se $n > 1$, siga os seguintes passos:

1. Mova os $n-1$ discos superiores da haste de origem para a haste auxiliar, usando a haste de destino como auxílio.
2. Mova o maior disco (n -ésimo disco) da haste de origem para a haste de destino.
3. Mova os $n-1$ discos da haste auxiliar para a haste de destino, usando a haste de origem como auxílio.

A função recursiva para a Torre de Hanoi pode ser definida em termos de si mesma, com o argumento $n-1$. Essa definição recursiva simplifica o processo de resolução da Torre de Hanoi e proporciona uma base sólida para a compreensão de outros exemplos que envolvem recursividade.

Em Python, a função recursiva da Torre de Hanoi ficaria assim:

```
def hanoi(n, origem, auxiliar, destino):  
    if n == 1:  
        print(f"Mova o disco 1 de {origem} para {destino}")  
    else:  
        hanoi(n - 1, origem, destino, auxiliar)  
        print(f"Mova o disco {n} de {origem} para {destino}")  
        hanoi(n - 1, auxiliar, origem, destino)
```

Neste código, a função `hanoi(n, origem, auxiliar, destino)` verifica se n é igual a 1 (caso base) e, nesse caso, imprime a instrução para mover o disco diretamente para a haste de destino. Se n for maior que 1, a função chama a si mesma com o argumento $n - 1$ (passo recursivo), seguindo os passos descritos anteriormente.

Fractais

Curva de Koch

Neste exemplo, vamos desenhar uma curva de Koch, que é um fractal simples e fácil de entender. Para criar a curva de Koch, seguiremos os seguintes passos:

- Comece com uma linha reta.
- Divida a linha em três partes iguais.
- Remova o segmento do meio e substitua-o por outros dois segmentos da mesma medida, formando um triângulo equilátero com a base no segmento removido.
- Repita este processo recursivamente para cada segmento de linha.

Para desenhar a curva de Koch, usaremos a biblioteca `Turtle` do Python para criar uma função recursiva chamada `curva_koch`.

```
import turtle

def curva_koch(tartaruga, ordem, tamanho):
    if ordem == 0:
        tartaruga.forward(tamanho)
    else:
        for angulo in [60, -120, 60, 0]:
            curva_koch(tartaruga, ordem - 1, tamanho / 3)
            tartaruga.left(angulo)

# Configurações iniciais da Turtle
tela = turtle.Screen()
tartaruga = turtle.Turtle()
tartaruga.speed(0)
tartaruga.penup()
tartaruga.goto(-150, 100)
tartaruga.pendown()

# Desenhe a curva de Koch com uma ordem específica e tamanho
ordem = 3
tamanho = 300
curva_koch(tartaruga, ordem, tamanho)

tela.mainloop()
```

Vamos entender o código passo a passo:

1. Importamos a biblioteca `Turtle` e criamos a função `curva_koch` que recebe três argumentos: a tartaruga (`tartaruga`), a ordem do fractal (`ordem`) e o tamanho do segmento de linha (`tamanho`).
2. Dentro da função `curva_koch`, verificamos se a ordem do fractal é 0. Se for, desenhamos uma linha reta com o tamanho especificado usando `tartaruga.forward(tamanho)`.
3. Se a ordem do fractal não for 0, percorremos uma lista de ângulos `[60, -120, 60, 0]` e chamamos a função `curva_koch` recursivamente com a ordem decrementada e o tamanho dividido por 3. A cada chamada recursiva, giramos a tartaruga para a esquerda pelo ângulo especificado usando `tartaruga.left(angulo)`.
4. Configuramos a tela e a tartaruga com as posições iniciais e velocidade.
5. Chamamos a função `curva_koch` com uma ordem específica (neste caso, 3) e um tamanho específico (neste caso, 300).
6. Finalmente, chamamos `tela.mainloop()` para manter a janela aberta.

Ao executar este código, você verá a curva de Koch sendo desenhada na tela com a ordem e o tamanho especificados. A técnica de recursividade é aplicada para criar o padrão fractal, repetindo o processo de subdivisão e formação de triângulos em cada segmento de linha. Experimente mudar alguns de seus argumentos, especialmente os que são utilizados na recursividade, para verificar o que acontece.

Triângulo de Sierpinski

O triângulo de Sierpinski é um fractal que consiste em dividir um triângulo equilátero em quatro triângulos equiláteros menores, removendo o triângulo central e repetindo o processo recursivamente nos três triângulos restantes.

Vamos usar mais uma vez a biblioteca `Turtle` do Python para criar uma função recursiva chamada `triangulo_sierpinski`.

```
import turtle

def triangulo_sierpinski(tartaruga, ordem, tamanho):
    if ordem == 0:
        for _ in range(3):
            tartaruga.forward(tamanho)
```



```

        tartaruga.left(120)
    else:
        triangulo_sierpinski(tartaruga, ordem - 1, tamanho / 2)
        tartaruga.forward(tamanho / 2)
        triangulo_sierpinski(tartaruga, ordem - 1, tamanho / 2)
        tartaruga.backward(tamanho / 2)
        tartaruga.left(60)
        tartaruga.forward(tamanho / 2)
        tartaruga.right(60)
        triangulo_sierpinski(tartaruga, ordem - 1, tamanho / 2)
        tartaruga.left(60)
        tartaruga.backward(tamanho / 2)
        tartaruga.right(60)

# Configurações iniciais da Turtle
tela = turtle.Screen()
tartaruga = turtle.Turtle()
tartaruga.speed(0)
tartaruga.penup()
tartaruga.goto(-150, 150)
tartaruga.pendown()

# Desenhe o triângulo de Sierpinski com uma ordem específica e
tamanho
ordem = 4
tamanho = 300
triangulo_sierpinski(tartaruga, ordem, tamanho)

tela.mainloop()

```

Vamos entender o código passo a passo:

1. Importamos a biblioteca `Turtle` e criamos a função `triangulo_sierpinski` que recebe três argumentos: a tartaruga (`tartaruga`), a ordem do fractal (`ordem`) e o tamanho do triângulo (`tamanho`).
2. Dentro da função `triangulo_sierpinski`, verificamos se a ordem do fractal é 0. Se for, desenhamos um triângulo equilátero com o tamanho especificado.

3. Se a ordem do fractal não for 0, chamamos a função `triangulo_sierpinski` recursivamente com a ordem decrementada e o tamanho dividido por 2 para cada um dos três triângulos. Após cada chamada recursiva, ajustamos a posição da tartaruga de acordo com as necessidades.
4. Configuramos a tela e a tartaruga com as posições iniciais e velocidade.
5. Chamamos a função `triangulo_sierpinski` com uma ordem específica (neste caso, 4) e um tamanho específico (neste caso, 300).
6. Finalmente, chamamos `tela.mainloop()` para manter a janela aberta.

Ao executar este código, você verá o triângulo de Sierpinski sendo desenhado na tela com a ordem e o tamanho especificados.

Árvore de recursão

Por fim, outro exemplo de fractal que utiliza recursividade é a chamada "Árvore de recursão". Essa estrutura fractal simula o crescimento de uma árvore ramificada através de um processo recursivo. Vamos criar uma função chamada `arvore_recursao` para desenhar esse fractal.

```
import turtle

def arvore_recursao(tartaruga, comprimento, angulo):
    if comprimento < 5:
        return
    else:
        tartaruga.forward(comprimento)
        tartaruga.left(angulo)
        arvore_recursao(tartaruga, comprimento * 0.6, angulo)
        tartaruga.right(angulo * 2)
        arvore_recursao(tartaruga, comprimento * 0.6, angulo)
        tartaruga.left(angulo)
        tartaruga.backward(comprimento)

# Configurações iniciais da Turtle
tela = turtle.Screen()
tartaruga = turtle.Turtle()
tartaruga.speed(0)
tartaruga.penup()
tartaruga.goto(0, -200)
tartaruga.pendown()

# Desenhe a árvore de recursão
comprimento = 200
angulo = 30
arvore_recursao(tartaruga, comprimento, angulo)

tela.mainloop()
```

Vamos entender o código passo a passo:

1. Importamos a biblioteca `Turtle`.
2. Criamos a função `arvore_recurcao` que recebe três argumentos: a tartaruga (`tartaruga`), o comprimento inicial do ramo (`comprimento`) e o ângulo de ramificação (`angulo`).
3. Dentro da função `arvore_recurcao`, verificamos se o comprimento é menor que 5. Se for, retornamos sem desenhar nada (condição de parada).
4. Caso o comprimento seja maior ou igual a 5, avançamos com a tartaruga no comprimento especificado, viramos à esquerda no ângulo especificado e chamamos recursivamente a função `arvore_recurcao` com um comprimento reduzido em 40% (0.6 do comprimento original) e o mesmo ângulo.
5. Em seguida, viramos à direita no dobro do ângulo e chamamos novamente a função `arvore_recurcao` com o mesmo comprimento reduzido e o mesmo ângulo.
6. Por fim, viramos à esquerda no ângulo especificado, voltamos com a tartaruga no comprimento especificado (voltando à posição inicial do ramo) e a recursão é concluída para esse ramo.
7. Configuramos a tela e a tartaruga com as posições iniciais e velocidade.
8. Chamamos a função `arvore_recurcao` com o comprimento inicial (neste caso, 200) e o ângulo de ramificação (neste caso, 30) para desenhar a árvore de recursão.
9. Finalmente, chamamos `tela.mainloop()` para manter a janela aberta.

Ao executar esse código, você verá uma representação de uma árvore ramificada sendo desenhada na tela usando a recursividade. O comprimento e o ângulo podem ser ajustados para obter diferentes formas e tamanhos de árvores.